Vouch for the Patch, Not the Person

Potency: a Gemma 4 harness that ranks the files tied to an edit by a joined import and commit-history graph, and refuses submission until each of the top five is changed or explained to the maintainer

Abstract

Open-source projects are closing to outside contributions: merged pull requests on GitHub rose 3.6 times from 2023 to 2026, and curl, tldraw and Ghostty now filter submissions because they are costly to check. One part of a patch is cheap to check. Seventeen public harnesses edit only some of the reference fix's files in 70–72% of multi-file SWE-bench Verified runs, and a repository records which files belong together in its imports and its commit history. Potency, an open harness for Gemma 4, joins both into one file graph. At the agent's first submit it lists the five files the graph ranks closest to the edit and refuses the submission until each is changed or explained in one line; the answers become a report for the maintainer. On 780 incomplete patches from 17 public harnesses, the list names a missed file in 58% of runs against 40% for issue-text search (paired difference +0.18, 95% CI [0.05, 0.32] and [0.06, 0.31] in two sets), and its point estimate is higher in all 17 harnesses. A multi-step walk over the graph does no better than the one-step ranking. Live with Gemma 4 31B and graded by the SWE-bench tests, the check changed no outcome (10 of 31 patches resolve before and after it) while the agent answered 150 of 155 listed files, 148 with reasons: it changes the report, not the patch. We release the harness, a 780-row missing-file benchmark and scripts that recompute every replay number. No maintainer has rated the report yet.

1. The strain, and what a project can check

GitHub reports about 25 million merged pull requests a month in January 2023 and over 90 million in 2026, while reviewing one "still takes a human about as long as it ever did" (all pull requests, not only agent-written ones). Agent-written bug fixes are accepted 71% of the time against 88% for human ones (MSR 2026).

Maintainers have answered by filtering people. curl closed its bug bounty in January 2026 after the confirmed-report rate fell below 5%. tldraw began "automatically closing pull requests from external contributors". Ghostty auto-closes pull requests from contributors nobody has vouched for. A survey counts 281 project policies on AI contributions (arXiv:2609.07542).

Identity is cheap to check and a patch is not. Our thesis is that one part of a patch can be checked as cheaply: a repository's imports and history rank the files most likely to belong to a change, and a patch either accounts for those files or does not. Computing the list needs no model; the reasons are the agent's own words, and the maintainer judges them.

The check targets a measured failure. On the 71 SWE-bench Verified tasks whose reference fix edits two or more source files, 17 public harnesses resolve less than half as often as on single-file tasks, and 70–72% of their runs edit some but not all reference files (Table 1). Agents identify the faulty file in 72–81% of SWE-bench Lite trajectories (Majgaonkar et al. 2025); they stop early, on a signal the model controls in eight of ten audited harnesses (Yetukuri 2026), and completion gates in which the same model rechecks its work move accuracy by −0.5 to +5.3 points (Zhang et al. 2026).

	Set A (8 harnesses)	Set B (9 open-weight)
Resolve, multi-file / single-file tasks	0.30 / 0.67	0.25 / 0.62
Multi-file runs that edit some but not all reference files ("partial")	72%	70%
Resolve when partial / when every reference file is edited	0.26 / 0.60	0.22 / 0.54

Table 1. From the published predictions of 17 harnesses. Set A: Agentless, AutoCodeRover, SWE-agent, OpenHands, Lingxi, SWE-Exp, FrogBoss, Sonar. Set B: nine open-weight models (Kimi K2, Qwen3-Coder, Devstral, GLM 4.5/4.6, others).

2. Potency

Potency takes one issue to one patch plus a report. It is a single agent with shell, read, edit and submit tools (the SWE-agent contract, Yang et al. 2024), configured for Gemma 4 31B. We changed four layers of the harness and left two alone.

Layer	What Potency does	Evidence in this paper
Context before acting	ADK agent: thinking capped at 1,024 tokens, repository test command in the prompt. The live run in §6.3 uses mini-swe-agent's prompt with no cap	Earlier Gemma 4 runs on four L4s, 45 paired tasks: same 14 resolved, mean time 322 to 305 s; a 512-token cap lost 3
Tools	`complete`: given the working tree, lists files the joined graph ties to the edit	Built and tested; not measured alone
Planning	Unchanged. A staged locate-fix-finalize split won 2 and lost 3 of 41 paired tasks and ran 396 against 292 s	Negative result that set the default
Stopping	Binding check: submit is refused until each listed file is edited or has a written reason (§4)	§6.1, §6.3
Context trimming	Unchanged	None
Output	Change report built from the binding check's answers; Gemma 4 E4B marks each reason HOLDS or DOUBT (§4)	§6.3; no maintainer study

Table 2. Harness layers.

On inference-time compute, extra graph hops buy nothing (§6.2), and the check spends agent steps where the repository points.

3. A file graph that joins imports with history

Nodes are the source files at the issue's base commit. **Import edges** link two files when one imports the other, parsed from the Python AST. **History edges** link files f and g that changed in the same commit, weighted by c(f, g) / n(f): the share of f's commits that also touched g, over the last 3,000 non-merge commits before the base commit, skipping commits of more than 30 files. The fix commit is never seen. **Seeds** are the non-test files in the agent's patch.

One step, fused (the default). Three rankings are computed from the seeds: files one import edge away, ordered by number of seed links; files by summed history weight Σ c(s, g) / n(s) over seeds s; and BM25 of the issue text over files. Reciprocal rank fusion (k = 60; Cormack et al. 2009) merges them. Seeds and test files are dropped and the top 5 are kept.

Multi-step walk. A personalized PageRank restarts at the seeds and moves along both edge types at once, with transition weight c(f, g) / n(f) plus 1 / deg(f) for import neighbours, row-normalised, for 30 iterations. It can reach a file two hops away that no seed imports or has changed with. We test restart probabilities 0.15 and 0.5, alone and fused with BM25.

Both are computed outside the model and handed to it as a short list with evidence, not as an encoded graph (Fatemi, Halcrow and Perozzi 2024).

4. The binding check and the change report

An earlier advisory version only listed the files; in 22 Gemma 4 runs the agent opened them and resubmitted, recording nothing about the files it left alone. The binding version turns each refusal to edit into a sentence.

At the first submit, the harness sends one message. It lists each expected file with its evidence, for example "changed with `operations.py` in 8 of its last 17 commits", and asks the agent to open each file, grep it for the names it changed, and either change it or write `REASON <path>: <why no change is needed>`. A resubmission that leaves a file unanswered is refused with the list of open files, at most twice, with 10 more steps each time.

The **change report** is rendered from the same data with no model call: one row per expected file with its evidence and status (Figure 1). `python -m potency.report` produces it for any patch on any Python repository and exits non-zero while a file is unanswered, so a project can run it in CI on incoming pull requests whoever wrote them.

A reason is the agent's own claim. `potency/audit.py` sends each one to a second, small model, Gemma 4 E4B, with the issue, the diff, and the lines of the skipped file that mention the names the diff changes. E4B answers HOLDS or DOUBT with a short note printed beside the reason. We serve E4B on one L40S through any OpenAI-compatible server.

5. A missing-file benchmark

The replay yields a task that Park et al. (2017) posed on human patches and that, to our knowledge, no benchmark poses on agent patches: *given an agent's incomplete patch, name a file it missed*. Each of the 780 rows of `bench/missing_file.jsonl` is one unresolved run that edited a source file and missed a reference file: harness, SWE-bench instance, repository, base commit, the files the agent edited, and the reference files it missed. `bench/score.py` reports the share of rows with a missed file in a method's top k. The rows cover 69 tasks in 10 Python repositories; intervals resample tasks. Unlike issue-to-file localization (LocAgent, RepoGraph), the input includes the agent's edit, and the errors are real, not synthetic deletions.

6. Results

6.1 Replay on 17 harnesses

For each such run we rebuild the graph at the base commit, seed it with the run's own edited files, and ask whether the top 5 contain a reference file the run missed.

Ranking (top 5)	Set A	Set B
Imports	0.342	0.350
History	0.362	0.378
Issue BM25	0.398	0.399
One-step fusion of all three	**0.582**	**0.580**
Fusion − BM25, 95% CI over tasks	+0.184 [0.048, 0.323]	+0.181 [0.055, 0.308]
Runs / tasks	354 / 64	426 / 69

Table 3. Share of unresolved runs missing a reference file where the ranking names one. 4,000 task-clustered bootstrap resamples.

Fusion is above BM25 in each of the 17 harnesses (point estimates), with per-harness rates from 0.53 to 0.65 in set A and 0.48 to 0.63 in set B. It is 0.58 on OpenHands with Claude Opus 4.5, which resolves 78% of all tasks.

6.2 One step against a multi-step walk

Method (top 5)	Set A	Set B
Walk, restart 0.15	0.325	0.347
Walk, restart 0.5	0.384	0.415
Walk 0.15 + BM25	0.534	0.585
Walk 0.5 + BM25	0.585	0.624
One-step fusion (Table 3)	0.582	0.580
Walk 0.5 + BM25 − one-step, 95% CI	+0.003 [−0.112, +0.108]	+0.045 [−0.044, +0.138]

Table 4. Same runs as Table 3; 2,000 task-clustered resamples.

The walk alone trails one-step fusion by 0.16 to 0.26 (intervals excluding zero) and is close to issue BM25; fused with BM25 it is not distinguishable from one-step fusion. Our untested reading: missed files sit one edge from the edit, and longer walks spread weight onto hubs. One-step fusion stays the default.

6.3 Live binding check with Gemma 4

Setup. Gemma 4 31B (QAT, w4a16) on vLLM 0.19.1, one H100, drives mini-swe-agent 2.4.6 on the 71 multi-file tasks in their SWE-bench containers: 60 steps, temperature 1.0, 4,096 tokens per turn, then the binding message at the first submit and 20 more steps. `live/grade_modal.py` grades each first and final patch with the task's SWE-bench tests in a Modal sandbox built from the prebuilt evaluation image; the reference patches resolve 31 of 31 graded tasks under it. We report the runs that finished before we stopped the clients.

Live binding run	Runs
Finished / with a submitted patch	49 / 31
Resolved by the tests: first patch / final patch	10 / 10
First patch covers every reference file / misses one	5 / 26
Missed files the check listed: edited / left with a reason / unanswered	1 / 11 / 0
All listed files: edited / left with a reason / unanswered	2 / 148 / 5
Reasons on reference files: run resolves / does not	2 / 9
E4B doubts: those 2 / those 9 / reasons on other files	0 / 3 / 23 of 137

Table 5. Coverage is measured against the reference patch. Of the 18 finished runs without a patch, eight hit the step limit, seven ended on malformed replies, two submitted an empty patch and one container failed to start. The five unanswered files come from one run that hit the step limit after the check.

The check changed no outcome: the final patch resolves exactly the tasks the first one did, none gained and none lost. Gemma 4 answers with reasons, not edits, and was never refused; of the twelve missed files the check listed, it edited one and explained eleven away. The reference patch is a poor judge of those reasons: two of the eleven come from runs that pass the tests, so the fix did not need that file. Figure 1 is one: the agent wrote that `views.py` needs nothing, and the tests agree. E4B doubted three of the other nine and neither of these two. Over all 148 reasons it doubted 18 of 104 in unresolved runs and 8 of 44 in resolved runs (Fisher p = 1.00), so it does not yet separate them; at temperature 0 a rerun repeats all 93 earlier verdicts.

Expected by the repository	Evidence	Status
`admindocs/views.py`	changed with `utils.py` in 1 of its last 5 commits; linked by an import	left alone: “It calls trim_docstring, but the fix is internal to that function and doesn’t change its signature or expected output format.” E4B: HOLDS
`admindocs/middleware.py`	same two links	left alone: “Does not use trim_docstring.”
`utils/regex_helper.py`	linked by an import	left alone: “Does not use trim_docstring.”

Figure 1. Change report for django__django-12155 (3 of 5 rows). Edited: `django/contrib/admindocs/utils.py`. The patch resolves the task.

7. Related work

Work	Signal	Where it acts	Evaluated on agent patches
ROSE (Zimmermann et al. 2005)	Co-change rules	IDE warnings on incomplete changes	No; human commits
Supplementary patches (Park et al. 2017)	Clones, dependencies, co-change	Locating the follow-up fix	No; human patches
Completion gates (Zhang et al. 2026)	The model’s own recheck	Submit	Yes; no repository signal
LocAgent, RepoGraph, Repository Memory (Wang et al. 2025)	Code graph, commit history	Localization, before editing	Localization accuracy only
Vouch; project AI policies	Contributor identity	Before review	Not applicable
Agent Governance Manifest (arXiv:2607.15769)	Declared contributor evidence	Pull request, for the reviewer	No measurement on agent patches
Potency	Imports + history + issue text, seeded on the agent’s edit	Submit, binding; report to the maintainer	780 patches, 17 harnesses; live binding run

Table 6. Closest prior work.

We know of no harness that makes the repository's record binding at submit and writes the agent's answers into the reviewer's report, nor of a measurement of such a check across harnesses.

8. Limitations and falsifiers

No maintainer has read the report, so we claim nothing about review time or merge rates. The replay scores against the reference patch; a valid fix can use other files, so a hit is a proxy for a useful prompt. The live run uses one model and one scaffold, and has no control arm. The import graph is Python-only and the history edges need a full clone. On a complete patch the check still costs the agent five looks and the maintainer five lines.

The claims would be falsified by: a harness in which one-step fusion does not beat issue BM25; a larger graded run in which the binding check loses more resolved patches than it gains (here it lost and gained none); or maintainers rating reports with reasons no more useful than a bare diff.

9. Reproducibility

The repository (github.com/Nas01010101/potency, MIT) holds `potency/` (graph, gate, report, audit); the ADK agent and its standard-library `complete` skill; `replay/` (`fetch.sh`, `harness_premises.py` for Table 1, `walk.py` for Tables 3 and 4 and the benchmark); `bench/`; `live/` with the binding run, the grader (`grade_modal.py`), the E4B server and summary scripts; and 15 tests. The replay reads public SWE-bench predictions and runs on a laptop CPU. `walk.py` calls the shipped ranking code.

References

- Agent Governance Manifest. arXiv:2607.15769, 2026.
- Bug-Fixing in the Age of AI. MSR 2026. doi:10.1145/3793302.3793570.
- Chen et al. 2025. LocAgent. ACL 2025.
- Cormack, Clarke, Büttcher 2009. Reciprocal Rank Fusion. SIGIR.
- Fatemi, Halcrow, Perozzi 2024. Talk like a Graph. ICLR.
- Ghostty CONTRIBUTING.md; mitchellh/vouch, 2026.
- GitHub 2026. Pull request limits (blog, 18 June).
- Majgaonkar et al. 2025. Understanding Code Agent Behaviour. arXiv:2511.00197.
- Ouyang et al. 2025. RepoGraph. ICLR.
- Park, Kim, Bae 2017. Supplementary Patches in Open Source Projects. EMSE 22.
- Policies on AI contributions in open source. arXiv:2609.07542.
- Stenberg 2026. The end of the curl bug bounty. daniel.haxx.se, 26 January.
- tldraw 2026. Issue 7695, 15 January.
- Wang et al. 2025. Code Localization with Repository Memory. arXiv:2510.01003.
- Yang et al. 2024. SWE-agent. NeurIPS.
- Yetukuri 2026. Completion without Evidence. raizenai.com/blog/papers/completion-without-evidence.pdf, 22 August.
- Zhang et al. 2026. Rethinking Verification in Agent Harnesses. arXiv:2610.34577.
- Zimmermann et al. 2005. Mining Version Histories to Guide Software Changes. IEEE TSE 31(6).